

# CI1212 Arquitetura de Computadores - UFPR

## Especificação do Trabalho 2

O trabalho deverá ser desenvolvido **individualmente**. O aluno deverá desenvolver um projeto de implementação da arquitetura **REDUX-V**. Trata-se de uma arquitetura de 8 bits, do tipo *load-store*, com endereçamento por byte. A organização a ser implementada **poderá ser do tipo monociclo**.

A lista de instruções, bem como seus respectivos formatos, encontra-se na próxima página. A memória RAM utilizada deverá ser o componente **"dual\_port\_ram"** disponibilizado. O clock do processador deverá ser configurado para durar **2 ticks**, sob pena de funcionamento incorreto da memória.

Adicionalmente, a implementação deverá utilizar os *opcodes* reservados para a criação de, no mínimo, **três novas instruções**, definidas a critério do aluno.

Recomenda-se a seguinte sequência de desenvolvimento:

1. Elaboração do diagrama de blocos do caminho de dados (*datapath*) do processador.
2. Elaboração do diagrama de blocos da Unidade Lógica e Aritmética (ULA), incluindo o planejamento dos códigos de operação utilizados em cada função.
3. Implementação do projeto no Logisim Evolution, utilizando componentes e interconexões.
4. Reescrita do programa desenvolvido no Trabalho 1, incorporando as novas instruções criadas.

### Regras Gerais de Entrega

A implementação deverá ser realizada no **Logisim Evolution**, podendo ser utilizados todos os componentes disponíveis na ferramenta. Cada aluno deverá entregar tanto o diagrama do REDUX-V quanto o projeto implementado.

### Sobre os artefatos a serem entregues:

#### Relatório contendo:

- Datapath do processador
- Tabela de códigos da ULA
- Tabela de controle
- Código Assembly do novo programa no relatório.
- Justificativa para as instruções adicionadas, evidenciando sua utilidade.

#### Circuito feito no logisim evolution funcionando

- Entrega das memórias (ROM de controle e RAM) devidamente preenchidas.
- Eventualmente pode ser entregue um arquivo separado para preencher a memória RAM.

O prazo de entrega será **impreterivelmente às 6h (a.m.)** da data estipulada (atenção: não confundir com 18h). Casos omissos neste enunciado deverão ser discutidos com o professor. Os trabalhos são **individuais**. A constatação de plágio implicará atribuição de **nota zero a todos os envolvidos**.

### Apresentação

Os trabalhos deverão ser apresentados oralmente pelo aluno. A avaliação considerará:

- Domínio do conteúdo;
- Robustez da solução proposta;
- Rigor metodológico.

## Observação

<sup>1</sup> O trabalho deverá ser desenvolvido utilizando o **Logisim Evolution v. 4.1.0**:

<https://github.com/logisim-evolution/logisim-evolution/releases>

<sup>2</sup> O código asm e o binário não devem considerar a existencia de um montador. Logo os endereços de salto devem ser resolvidos pelo aluno.

# Descrição da Arquitetura REDUX-V

## Informações gerais

- 1. **Tamanho da palavra:** 8 bits
- 2. Arquitetura **Load-Store**
- 3. Memória endereçada a cada 1 Byte
- 4. Arquitetura **Von Neumann**

## Conjunto de instruções (ISA)

| Opcode | Tipo                              | Mnemonic | Nome                           | Operação                         |
|--------|-----------------------------------|----------|--------------------------------|----------------------------------|
| 0000   | R                                 | brzr     | Branch On Zero Register        | if (R[ra] == 0) PC = R[rb]       |
| 0001   | I                                 | ji       | Jump Immediate (signed)        | PC = PC + Imm.                   |
| 0010   | R                                 | ld       | Load                           | R[ra] = M[ <b>R[rb]</b> ]        |
| 0011   | R                                 | st       | Store                          | M[ <b>R[rb]</b> ] = R[ra]        |
| 0100   | I                                 | addi     | Add Immediate (signed)         | <b>R[0]</b> = <b>R[0]</b> + Imm. |
| 0101   | Reservado - Instruções adicionais |          |                                |                                  |
| 0110   |                                   |          |                                |                                  |
| 0111   |                                   |          |                                |                                  |
| 1000   | R                                 | not      | Not (logical)                  | R[ra] = not R[rb]                |
| 1001   | R                                 | and      | And (bitwise)                  | R[ra] = R[ra] and R[rb]          |
| 1010   | R                                 | or       | Or (bitwise)                   | R[ra] = R[ra] or R[rb]           |
| 1011   | R                                 | xor      | Xor (bitwise)                  | R[ra] = R[ra] xor R[rb]          |
| 1100   | R                                 | add      | Add (signed)                   | R[ra] = R[ra] + R[rb]            |
| 1101   | R                                 | sub      | Sub (signed)                   | R[ra] = R[ra] - R[rb]            |
| 1110   | R                                 | slr      | Shift Left Register (bitwise)  | R[ra] = R[ra] << R[rb]           |
| 1111   | R                                 | srr      | Shift Right Register (bitwise) | R[ra] = R[ra] >> R[rb]           |

| Tipo I |        |   |   |   |      |   |   |   |
|--------|--------|---|---|---|------|---|---|---|
| Bits   | 7      | 6 | 5 | 4 | 3    | 2 | 1 | 0 |
|        | Opcode |   |   |   | Imm. |   |   |   |

Formatos das instruções

| Tipo R |        |   |   |   |    |   |    |   |
|--------|--------|---|---|---|----|---|----|---|
| Bits   | 7      | 6 | 5 | 4 | 3  | 2 | 1  | 0 |
|        | Opcode |   |   |   | Ra |   | Rb |   |